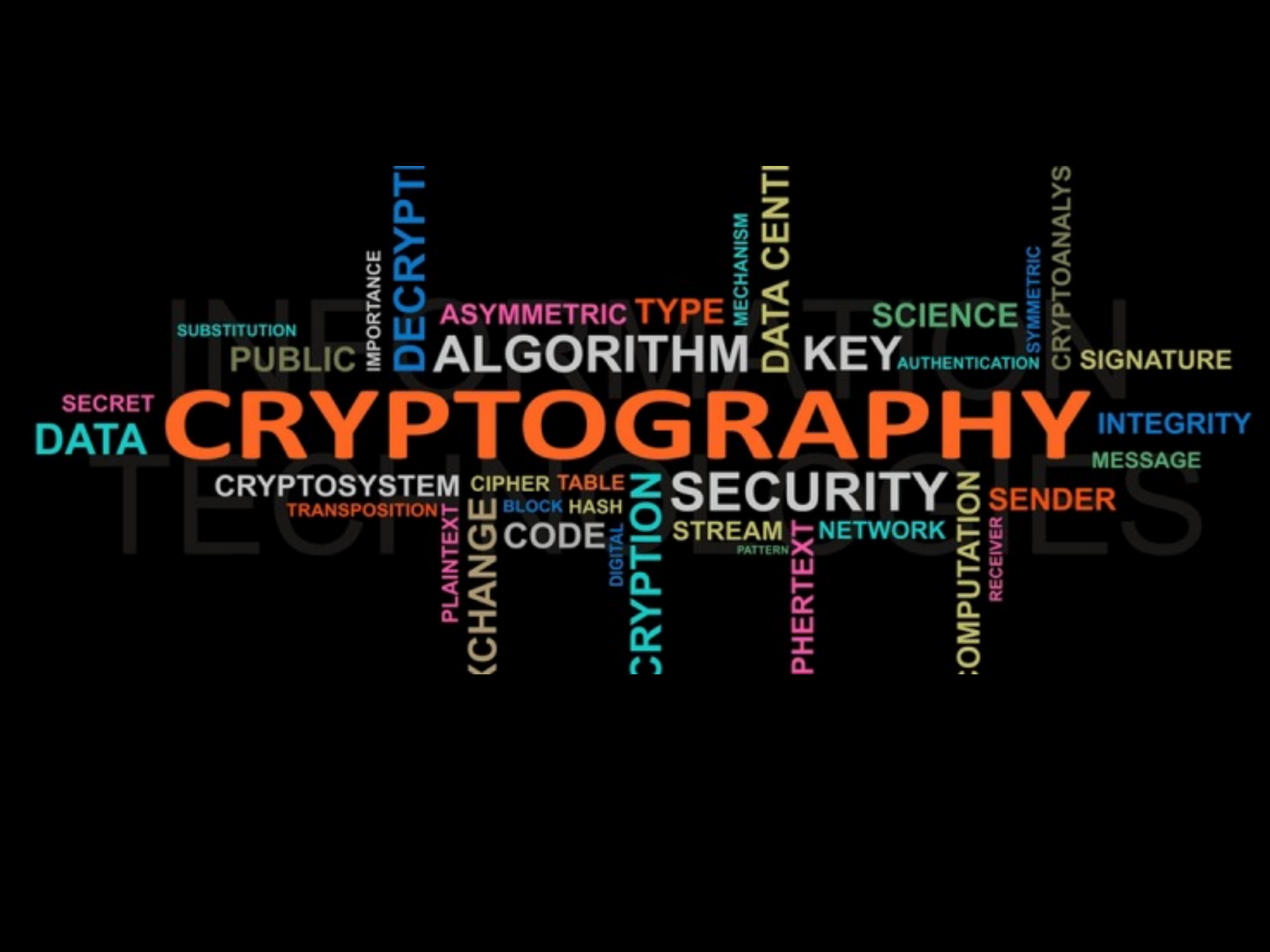

CRYPTOGRAPHY
ALGORITHM
SECURITY
KEY
DATA
SECRET
PUBLIC
SUBSTITUTION
IMPORTANCE
DECRYPTI
ASYMMETRIC
TYPE
MECHANISM
DATA CENTI
SCIENCE
SYMMETRIC
CRYPTOANALYS
AUTHENTICATION
SIGNATURE
INTEGRITY
MESSAGE
CRYPTOSYSTEM
CIPHER
TABLE
TRANSPOSITION
BLOCK
HASH
CODE
STREAM
NETWORK
PATTERN
SENDER
RECEIVER
PLAINTEXT
DIGITAL
CRYPTION
PHERTEXT
COMPUTATION

Защита информации

Ирина Сергеевна

Кафедра ПМиК:
430а (главный корпус),
406 (новый корпус)

Формы освоения материала

Лекции

Лабораторные работы

Расчетно-графическая работа

Формы контроля знаний

Контроль посещения лекций и практик

Защита лабораторных работ (оценка)

Контрольные работы

Защита РГР (оценка)

Тест

Подсчет баллов

Экзамен (оценка)

Рейтинг (баллы)

Лекции: посещение (2 балла), ответы (2 балла)

Лабораторные работы: оценки 3, 4, 5

Автомат:

- посещение лекций

  ≥50% от max (на оценку 3),

  ≥ 65% от max (на оценку 4),

  ≥ 80% от max (на оценку 5)
- все лабораторные и РГР сданы
- за обе контрольные и тест набраны проходные баллы

Экзамен по билетам: <50% от max посещения лекций, не сданы лабы, РГР или контрольные и тест написаны плохо (не набрали проходной балл)

Криптография:

- злоумышленник знает, что сообщение зашифровано и пытается его дешифровать

Стеганография:

- злоумышленник может не догадываться, что сообщение зашифровано

Проблема защиты информации

Проблема защиты информации от несанкционированного доступа обострилась в связи с широким распространением **локальных** и **глобальных** компьютерных сетей.

Защита информации необходима для уменьшения вероятности:

- утечки (разглашения) информации,
- модификации (умышленного искажения) информации,
- утраты (уничтожения) информации, представляющей определенную ценность для её владельца.

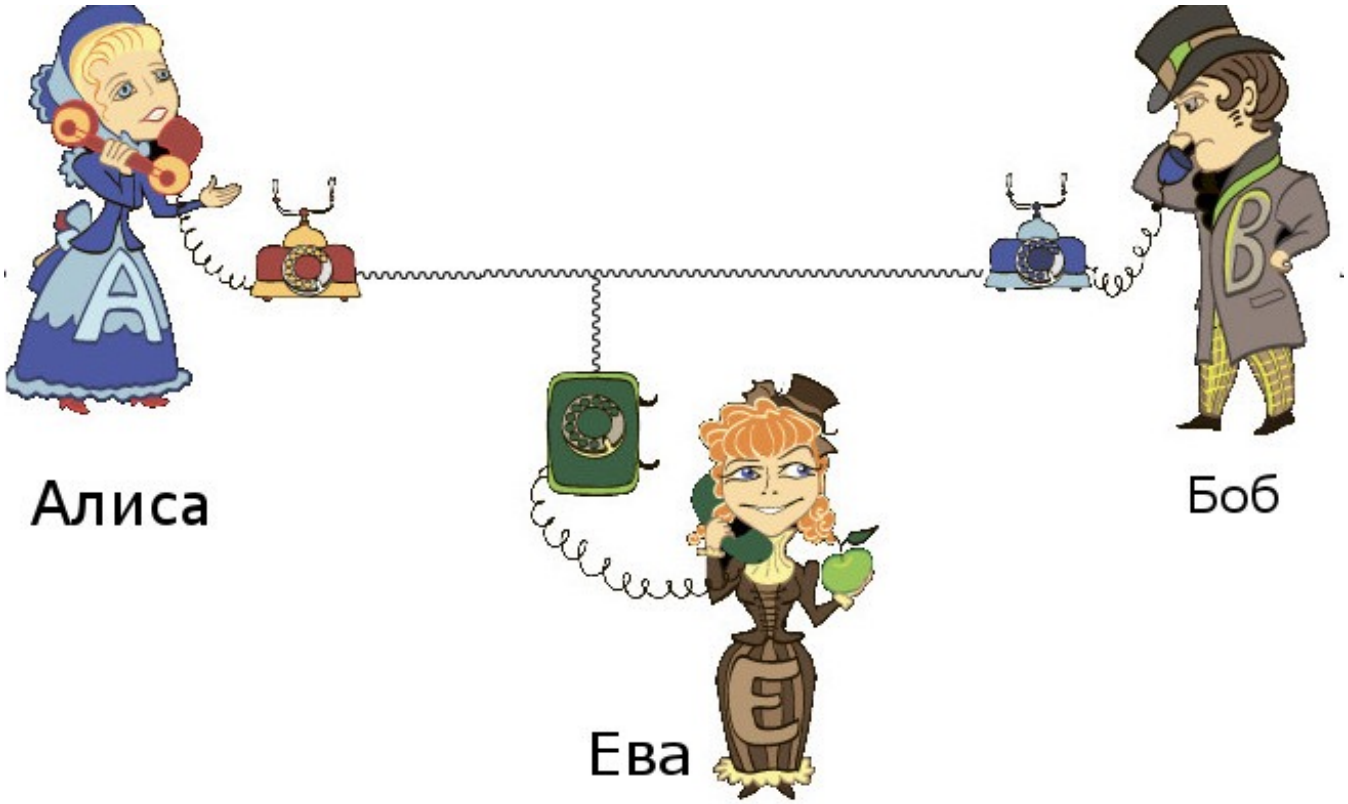
Должны быть защищены:

- межбанковские расчеты по компьютерным сетям
- биржи, в которых все расчеты проводятся через Интернет
- расчеты по кредитным картам
- перевод заработной платы в банк
- заказ билетов через Интернет
- покупки в Интернет-магазинах
- разговоры по мобильным телефонам и электронная почта и т.д.

Задача передачи секретных сообщений от отправителя A (Alice) к получателю B (Bob)

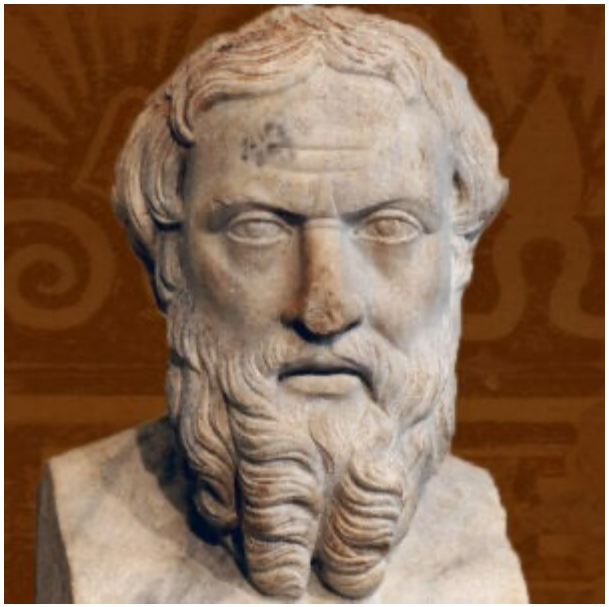
Отправитель сообщений и их получатель могут быть физическими лицами, организациями, какими-либо техническими системами.

Предполагается, что сообщения передаются по так называемому «открытому» каналу связи, доступному для прослушивания некоторым другим лицам, отличным от получателя и отправителя.



История возникновения шифров

Проблема защиты информации волнует людей несколько столетий.

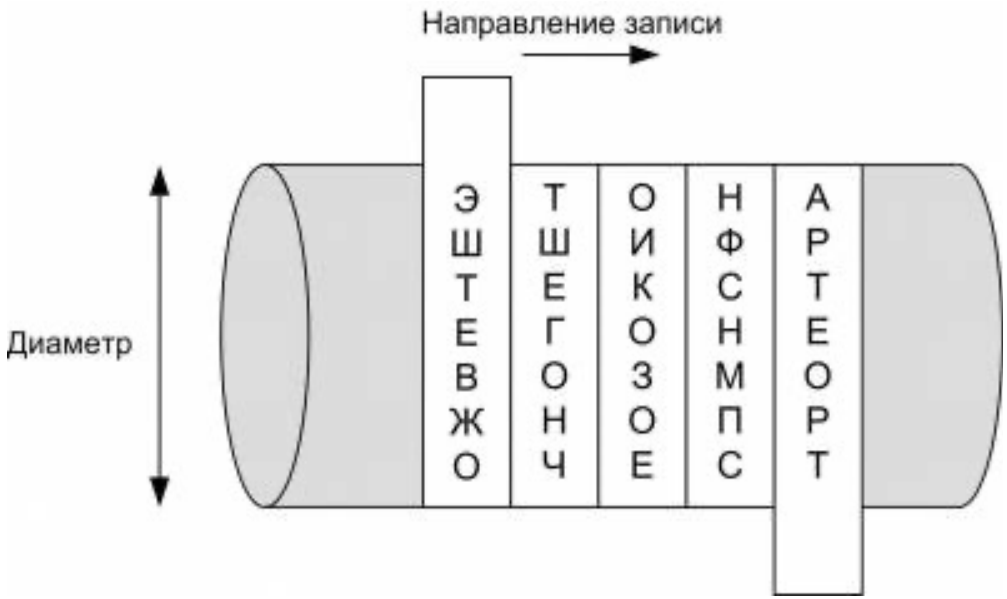


Геродот

По свидетельству Геродота,
уже в **V в. до н.э.** использовалось
преобразование информации
методом кодирования.

Одним из первых шифровальных приспособлений была **СКИТАЛА**, которая применялась в v в. до н.э. во время войны Спарты против Афин.

СКИТАЛА – это цилиндр, на который наматывалась узкая папирусная лента (без пробелов и нахлестов). Затем на этой ленте вдоль оси цилиндра (столбцами) записывался текст. Лента сматывалась с цилиндра и отправлялась получателю.



Получив такое сообщение, получатель наматывал ленту на цилиндр такого же диаметра, как и диаметр СКИТАЛЫ отправителя. В результате можно было прочитать зашифрованное сообщение.

Шифр перестановки "Скитала"

Исходное сообщение:

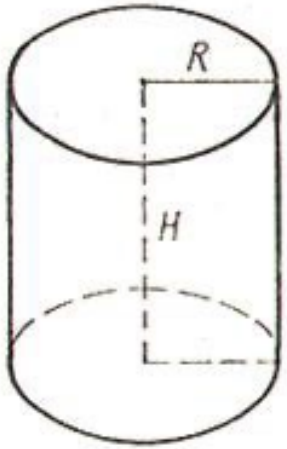
ЭТО НАШ ШИФРТЕКСТ, ЕГО НЕВОЗМОЖНО ПРОЧЕСТЬ

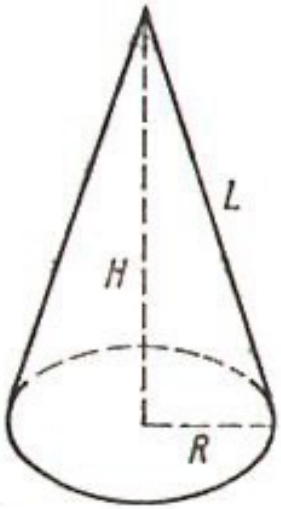
При использовании шифра «Скитала» получится:

ЭШТЕВ ЖОТШЕ ГОНЧО ИКОЗО ЕНФСН МПСАР ТЕОРТ.

Для расшифрования такого шифртекста **нужно** не только **знать правило шифрования**, но и **обладать ключом** в виде стержня определенного диаметра.

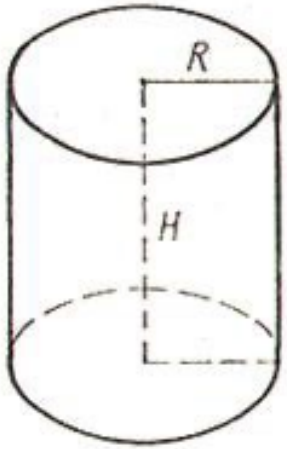
Аристотелю принадлежит идея **ВЗЛОМА** такого шифра.

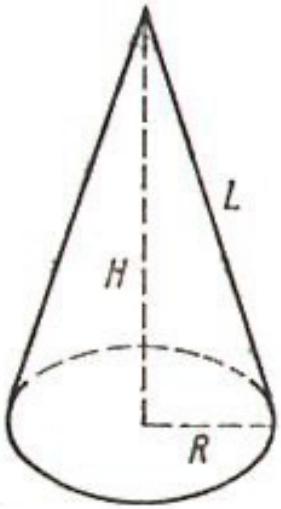




Он предложил изготовить длинный конус и, начиная с основания, обертывать его лентой с шифрованным сообщением, постепенно сдвигая её к вершине. На каком-то участке конуса начнут просматриваться участки читаемого текста. Так определяется секретный размер цилиндра.

Аристотелю принадлежит идея **ВЗЛОМА** такого шифра.





Он предложил изготовить длинный конус и, начиная с основания, обертывать его лентой с шифрованным сообщением, постепенно сдвигая её к вершине. На каком-то участке конуса начнут просматриваться участки читаемого текста. Так определяется секретный размер цилиндра.

Шифры появились в глубокой древности в виде криптограмм – «тайнопись» (греч.).

Священные иудейские тексты часто шифровались методом замены. Вместо первой буквы алфавита записывалась последняя буква, вместо второй – предпоследняя и т.д. Этот древний шифр назывался **АТБАШ**.

А	Б	В	Г	Д	Е	Ё	Ж	З	И	Й	К	Л	М	Н	О	П
Я	Ю	Э	Ь	Ы	Ъ	Щ	Ш	Ч	Ц	Х	Ф	У	Т	С	Р	П

Шифр Цезаря

Известен факт шифрования переписки

Юлия Цезаря (100-44 до н.э.)

с Цицероном (106-43 до н.э.).

Шифр Цезаря: каждая буква в сообщении заменяется другой буквой этого же алфавита, отстоящей от неё в алфавите на фиксированное количество букв (ключ шифрования).

В своих шифровках Цезарь заменял букву исходного открытого текста буквой, отстоящей от исходной буквы впереди на три позиции.

Например, слово ПЕРЕМЕНА после применения к нему шифра Цезаря превращается в ТИУИПИРГ (если исключить букву Ё и считать, что в алфавите 32 буквы).

Применение магических квадратов

В средние века для шифрования перестановкой применялись и «**магические квадраты**».

В общем случае ***магическим квадратом*** является расположение чисел от 1 до n^2 в виде квадрата так, что числа в каждом столбце, строчке и диагонали дают одинаковую сумму s, называемую ***магической суммой***.

17	24	1	8	15
23	5	7	14	16
4	6	13	20	22
10	12	19	21	3
11	18	25	2	9

Применение магических квадратов

Шифруемый текст вписывали в магические квадраты в соответствии с нумерацией их клеток.

Если затем выписать содержимое такой таблицы по строкам, то получится шифротекст, сформированный благодаря перестановке букв исходного сообщения.

В те времена считалось, что созданные с помощью магических квадратов шифротексты охраняет не только ключ, но и магическая сила.

Альбрехт Дюрер

"Меланхолия I", 1514 г.

Магическая сумма = 34

ЭТО КРИПТОГРАФИЯ

16	3	2	13
5	10	11	8
9	6	7	12
4	15	14	1

Я	О	Т	А
К	О	Г	П
Т	Р	И	Р
	И	Ф	Э

Шифротекст, получаемый при считывании содержимого правой таблицы по строкам, имеет вполне загадочный вид:

ЯОТА КОГП ТРИР ИФЭ

Квадрат Полибия

В Древней Греции был известен шифр, который создавался с помощью квадрата Полибия.

Таблица для шифрования – квадрат с пятью столбцами и пятью строками , которые нумеровались цифрами от 1 до 5. В каждую клетку записывалась одна буква.

В результате каждой букве соответствовала пара цифр, и шифрование сводилось к замене буквы парой цифр.

Порядок расположения символов в квадрате Полибия является секретной информацией (ключом).

	1	2	3	4	5
1	A	B	C	D	E
2	F	G	H	I	K
3	L	M	N	O	P
4	Q	R	S	T	U
5	V	W	X	Y	Z

Зашифруем с помощью квадрата Полибия слово КРИПТОГРАФИЯ:

26 36 24 35 42 34 14 36 11 44 24 63

	1	2	3	4	5	6
1	А	Б	В	Г	Д	Е
2	Ё	Ж	З	И	Й	К
3	Л	М	Н	О	П	Р
4	С	Т	У	Ф	Х	Ц
5	Ч	Ш	Щ	Ъ	Ы	Ь
6	Э	Ю	Я	,	.	-

Из примера видно, что в шифрограмме первым указывается номер строки, а вторым – номер столбца

Мы можем описать шифр Цезаря в общем виде, если пронумеруем (закодируем) буквы русского алфавита числами от 0 до 31 (исключив букву Ё). Тогда правило шифрования запишется следующим образом:

$$c = (m + k) \bmod 32, \quad (1.1)$$

где **m** и **c** – номера букв соответственно сообщения и шифротекста, а **k** – некоторое целое число, называемое ключом шифра (в рассмотренном выше шифре Цезаря **k = 3**).

Чтобы дешифровать зашифрованный текст, нужно применить «обратный» алгоритм:

$$\mathbf{m = (c - k) \bmod 32}. \quad (1.2)$$

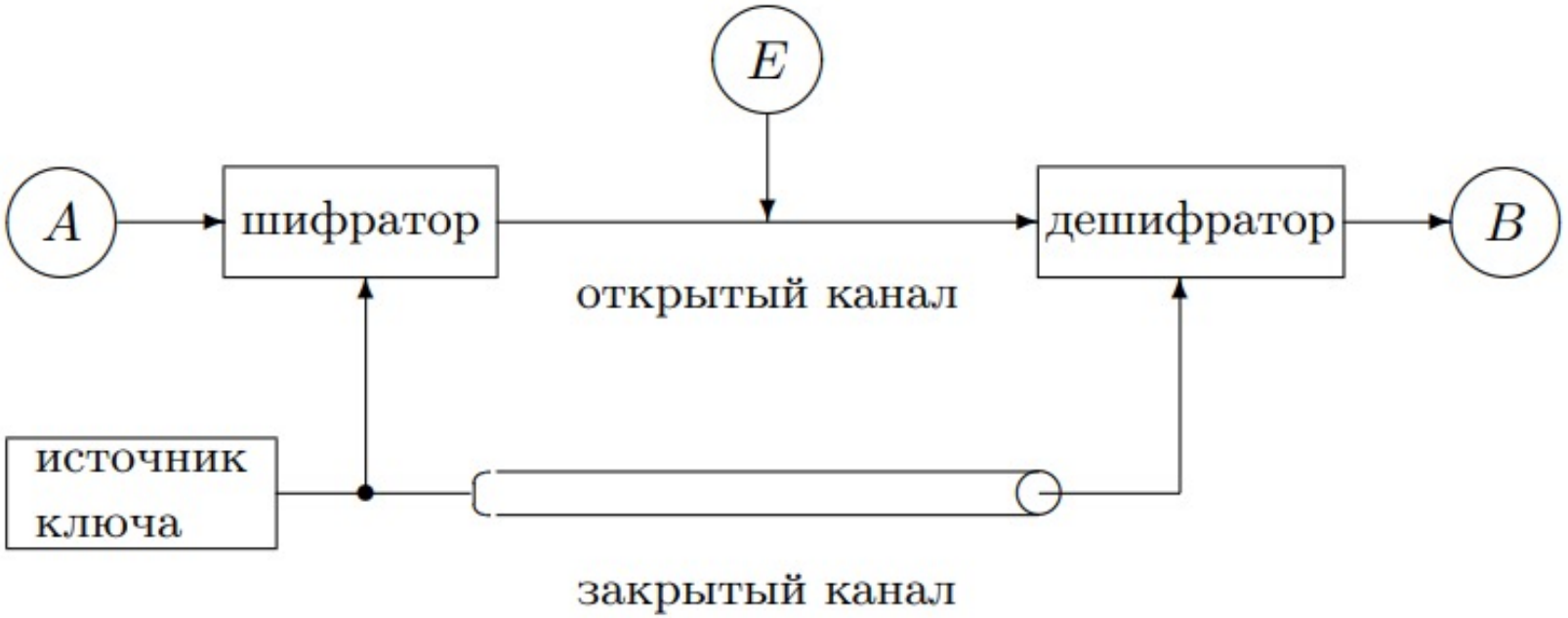


Рис. 1.1. Классическая система секретной связи

Попытки взлома шифра

В нашем примере Ева знает, что шифр был построен в соответствии с (1.1), что исходное сообщение было на русском языке и что был передан шифротекст **ТИУИПИРГ**, но ключ Еве не известен.

Наиболее очевидная попытка расшифровки – последовательный перебор всех возможных ключей (это так называемый метод «грубой силы» – ***brute-force attack***).

Ева перебирает последовательно все возможные ключи: **k = 1 , 2, . . . 32**, подставляя их в алгоритм дешифрования и оценивая получающиеся результаты.

Таблица 1.1. **Расшифровка слова ТИУИПИРГ путем перебора ключей**

k	m	k	m	k	m	k	m
1	СЗТ	9	ЙЯ	17	БЧ	25	ЩП
2	РЖС	10	ИЮЙ	18	АЦБ	26	ШОЩ
3	ПЕРЕМЕНА	11	ЗЭИ	19	ЯХА	27	ЧН
4	ОДП	12	ЖЬ	20	ЮФ	28	ЦМ
5	НГ	13	ЕЫ	21	ЭУ	29	ХЛЦ
6	МВ	14	ДЪ	22	Ь	30	ФК
7	ЛБМ	15	ГЩ	23	Ы	31	УЙ
8	КАЛАЗ	16	ВШГ	24	Ъ	32	ТИУИПИРГ

Видим, что был использован ключ **k = 3** и зашифровано сообщение **ПЕРЕМЕНА**.

Очевидно, что рассмотренный шифр **совершенно нестоек**! Для его вскрытия достаточно проанализировать несколько первых букв сообщения и после этого ключ **k** однозначно определяется (и однозначно дешифруется все сообщение).

В чем же причины нестойкости рассмотренного шифра и как можно было бы увеличить его стойкость?

Рассмотрим еще один пример. Алиса спрятала важные документы в ячейке камеры хранения, снабженной пятидекадным кодовым замком.

Теперь она хотела бы сообщить Бобу комбинацию цифр, открывающую ячейку. Она решила использовать аналог шифра Цезаря, адаптированный к алфавиту, состоящему из десятичных цифр:

$$c = (m + k) \bmod 10 \quad (1.3)$$

Допустим, Алиса послала Бобу шифротекст **26047**. Ева пытается расшифровать его, последовательно перебирая все возможные ключи. Результаты ее попыток сведены в табл. 1.2.

Таблица 1.2. **Расшифровка сообщения 26047 путем перебора ключей**

k	m	k	m
1	15936	6	60481
2	04825	7	59370
3	93714	8	48269
4	82603	9	37158
5	71592	0	26047

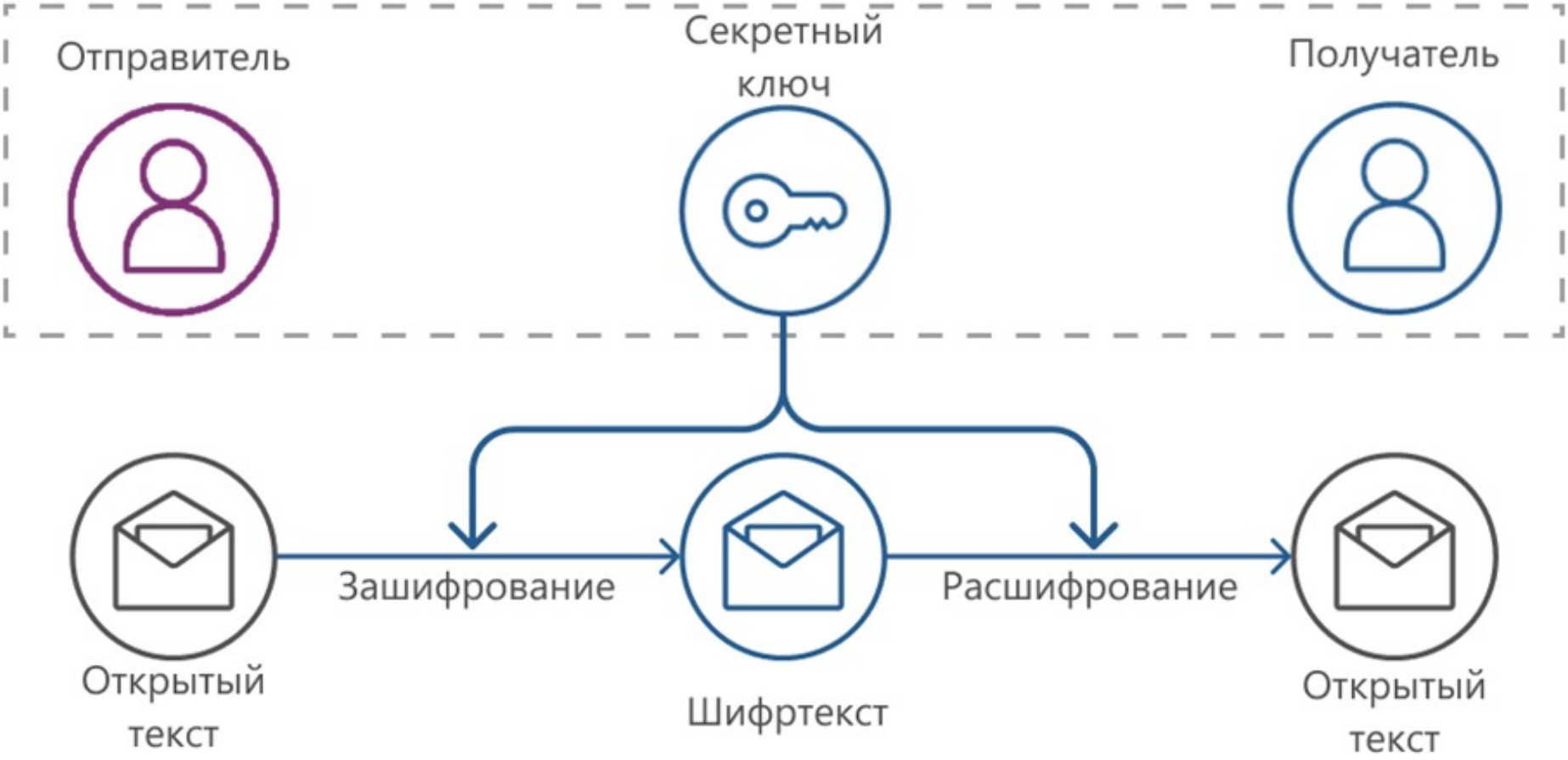
Все полученные варианты равнозначны и Ева не может понять, какая именно комбинация истинна.

Анализируя шифротекст, она не может найти значения секретного ключа.

Конечно, до перехвата сообщения у Евы было 10^5 возможных значений кодовой комбинации, а после — только 10.

Однако важно отметить то, что в данном случае всего 10 значений ключа. Поэтому при таком ключе (одна десятичная цифра) Алиса и Боб и не могли расчитывать на большую секретность.

Симметричное шифрование

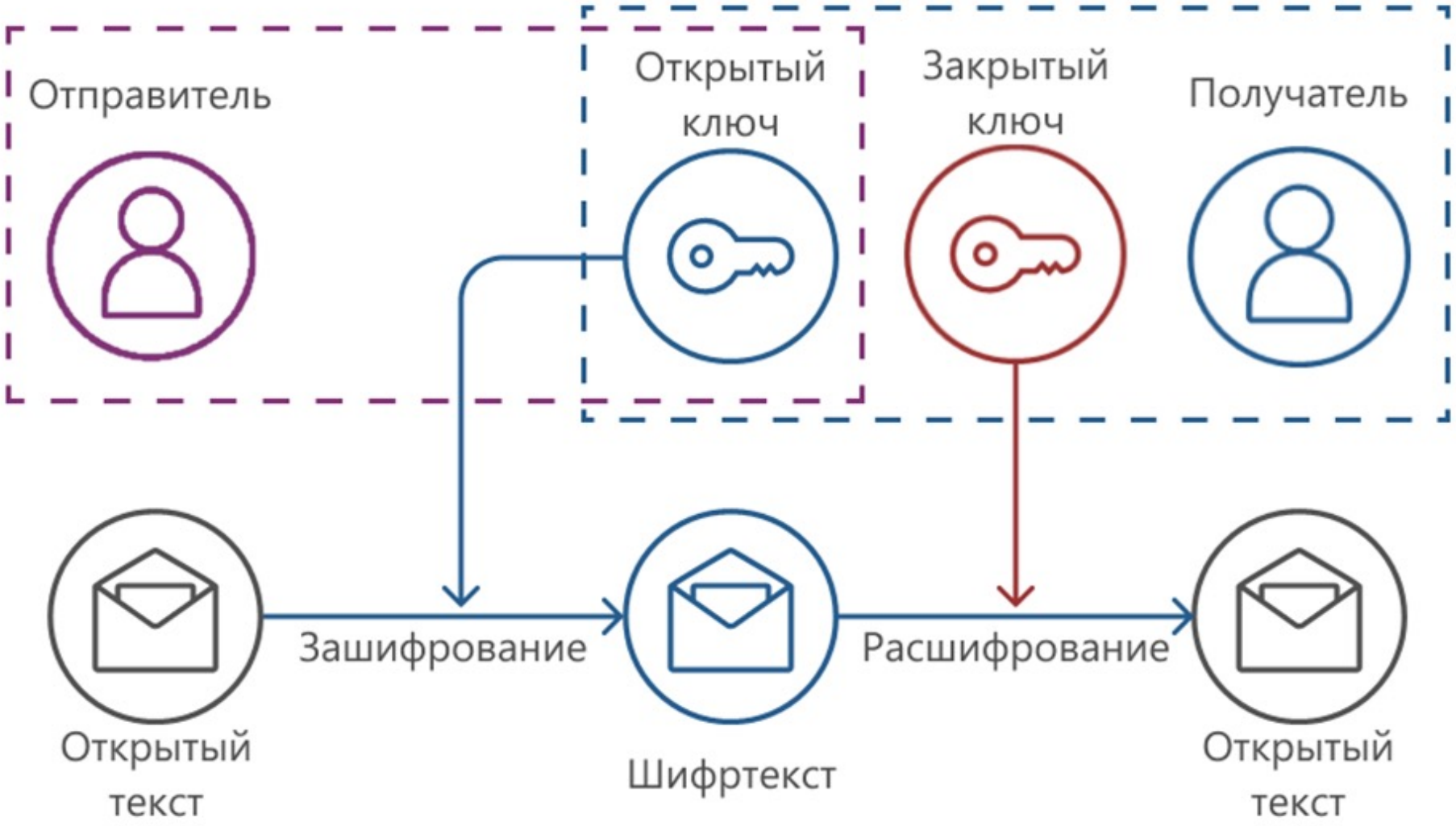


Примеры задач, решаемых криптографией с открытым ключом:

- Хранение паролей в компьютере.
- Верификация в системах ПВО (при пересечении самолетом границы радиолокатор спрашивает пароль).
- Удаленный доступ через компьютерные сети (например, при взаимодействии банка и клиента).

Обычно в начале сеанса банк запрашивает у клиента имя, а затем секретный пароль, но злоумышленник может узнать пароль, так как линия связи открытая.

Асимметричное шифрование



Сегодня все эти проблемы решаются с использованием криптографических методов. Решение всех этих задач основано на важном понятии **односторонней функции** (one-way function).

Определение 2.1. Пусть дана функция

$$y = f(x), \tag{2.1}$$

определенная на конечном множестве X ($x \in X$), для которой существует обратная функция

$$x = f^{-1}(y). \tag{2.2}$$

Функция называется *односторонней*, если вычисление по формуле (2.1) — простая задача, требующая немного времени, а вычисление по (2.2) — задача сложная, требующая привлечения массы вычислительных ресурсов, например, 10^6–10^{10} лет работы мощного суперкомпьютера.

В качестве примера односторонней функции рассмотрим следующую:

$$y = a^x \bmod p, \tag{2.3}$$

где p — некоторое простое число (т.е. такое, которое делится без остатка только на себя и на единицу), а x — целое число из множества $\{1, 2, \ldots, p-1\}$. Обратная функция обозначается

$$x = \log_a y \bmod p \tag{2.4}$$

и называется *дискретным логарифмом.*

Для того чтобы обеспечить трудность вычисления по (2.4) при использовании лучших современных компьютеров, в настоящее время используются числа размером **более 512 бит**. На практике часто применяются и другие односторонние функции, например, так называемые хеш-функции, оперирующие с существенно более короткими числами порядка 60–120 бит.

Быстрое возведение в степень

В качестве примера односторонней функции рассмотрим следующую:

$$y = a^x \bmod p, \tag{2.3}$$

где p — некоторое простое число (т.е. такое, которое делится без остатка только на себя и на единицу), а x — целое число из множества $\{1, 2, \ldots, p-1\}$.

Обратная функция обозначается

$$x = \log_a y \bmod p \tag{2.4}$$

и называется *дискретным логарифмом*.

$$y = a^x \bmod p$$

```
#include<stdlib.h>
#include<iostream>
#include<math.h>

using namespace std;


int main()
{
  int a,x,p,y;

  cin>>a>>x>>p;

 y=(int) pow(a,x)%p;

 cout<<y;


system("pause");
return 0;
}
```

НЕПОДХОДЯЩЕЕ РЕШЕНИЕ

Сначала мы покажем, что вычисление по (2.3) может быть выполнено достаточно быстро. Начнем с примера вычисления числа $a^{16} \bmod p$. Мы можем записать

$$a^{16} \bmod p = \left(\left((a^2)^2 \right)^2 \right)^2 \bmod p,$$

т.е.значение данной функции вычисляется всего за 4 операции умножения вместо 15 при «наивном» варианте $a \cdot a \cdot \ldots \cdot a$. На этом основан общий алгоритм.

Для описания алгоритма введем величину $t = \lfloor \log_2 x \rfloor$ — целую часть $\log_2 x$ (далее все логарифмы будут двоичные, поэтому в дальнейшем мы не будем указывать основание 2). Вычисляем числа ряда

$$a, \quad a^2, \quad a^4, \quad a^8, \quad \ldots, \quad a^{2^t} \qquad (\bmod p). \tag{2.5}$$

$$ab \bmod n = ((a \bmod n)(b \bmod n)) \bmod n$$

В ряду (2.5) каждое число получается путем умножения предыдущего числа самого на себя по модулю p. Запишем показатель степени x в двоичной системе счисления:

$$x = (x_t x_{t-1} \ldots x_1 x_0)_2.$$

Тогда число $y = a^x \bmod p$ может быть вычислено как

$$y = \prod_{i=0}^{t} a^{x_i \cdot 2^i} \bmod p \qquad (2.6)$$

(все вычисления проводятся по модулю p).

Пример 2.1. Пусть требуется вычислить $3^{100} \bmod 7$. Имеем $t = \lfloor \log 100 \rfloor = 6$. Вычисляем числа ряда (2.5):

$$\begin{array}{ccccccc} a & a^2 & a^4 & a^8 & a^{16} & a^{32} & a^{64} \\ 3 & 2 & 4 & 2 & 4 & 2 & 4 \end{array} \tag{2.7}$$

Записываем показатель в двоичной системе счисления:

$$100 = (1100100)_2$$

и проводим вычисления по формуле (2.6):

$$\begin{array}{ccccccccccc} a^{64} & & a^{32} & & & & a^4 & & & & \\ 4 & \cdot & 2 & \cdot & 1 & \cdot & 1 & \cdot & 4 & \cdot & 1 & \cdot & 1 & = & 4 \end{array} \tag{2.8}$$

Нам потребовалось всего 8 операций умножения (6 для вычисления ряда (2.7) и 2 для (2.8)).

Алгоритм 2.3. Возведение в степень (справа-налево)

ВХОД: Целые числа a, $x = (x_t x_{t-1} \dots x_0)_2$, p.
ВЫХОД: Число $y = a^x \bmod p$.

1. $y \leftarrow 1$, $s \leftarrow a$.
2. FOR $i = 0, 1, \dots, t$ DO
3. IF $x_i = 1$ THEN $y \leftarrow y \cdot s \bmod p$;
4. $s \leftarrow s \cdot s \bmod p$.
5. RETURN y.

Чтобы показать, что по представленному алгоритму действительно вычисляется y согласно (2.6), запишем степени переменных после каждой итерации цикла. Пусть $x = 100 = (1100100)_2$, как в примере 2.1, тогда:

i:	0	1	2	3	4	5	6
x_i:	0	0	1	0	0	1	1
y:	1	1	a^4	a^4	a^4	a^{36}	a^{100}
s:	a^2	a^4	a^8	a^{16}	a^{32}	a^{64}	a^{128}

В некоторых ситуациях более эффективным оказывается следующий алгоритм, в котором биты показателя степени просматриваются слева-направо, т.е. от старшего к младшему.

Алгоритм 2.4. Возведение в степень (слева-направо)

ВХОД: Целые числа a, $x = (x_t x_{t-1} \dots x_0)_2$, p.
ВЫХОД: Число $y = a^x \bmod p$.
1. $y \leftarrow 1$.
2. FOR $i = t, t-1, \dots, 0$ DO
3. $y \leftarrow y \cdot y \bmod p$;
4. IF $x_i = 1$ THEN $y \leftarrow y \cdot a \bmod p$.
5. RETURN y.

Чтобы убедиться в том, что алгоритм 2.4 вычисляет то же самое, что и алгоритм 2.3, запишем степени переменной y после каждой итерации цикла для $x = 100$:

i:	6	5	4	3	2	1	0
x_i:	1	1	0	0	1	0	0
y:	a	a^3	a^6	a^{12}	a^{25}	a^{50}	a^{100}

Проверка числа на простоту за O (sqrt(P))

Чтобы определить, является ли данное число P простым, безусловно, достаточно написать простой цикл поиска делителей числа P:

```
bool prime (long long p){
    for (long long i = 2; i <= sqrt(p); i++)
        if (p%i == 0)
            return false;
    return true;
}
```

Данная функция проверки числа на простоту достаточно эффективна. Однако иногда нужно уметь проверять число на простоту быстрее.

Проверка числа на простоту за O (K*log P)

Чтобы проверить число **P** на простоту с достаточно хорошей вероятностью безошибочности, достаточно **K** раз (например, 100 раз) проверить случайное число **a** **тестом Ферма**.

Теорема 2.6 (Ферма). *Пусть p — простое число и $0 < a < p$. Тогда*

$$a^{p-1} \bmod p = 1.$$

1. Если для основания a и модуля n выполняется $a^{n-1} \equiv 1 (mod\ n)$, то n может быть простым числом.
2. Если $a^{n-1} \not\equiv 1 (mod\ n)$, то n - однозначно составное.

При проверке числа на простоту тестом Ферма выбирают несколько чисел **a**. Чем больше количество a, для которых $a^{n-1} \bmod p = 1$, тем больше шансы, что число **p** - простое.

Однако существуют составные числа, для которых сравнение $a^{n-1} \bmod p = 1$ выполняется для всех **a**, взаимно простых с **p**.

Это **числа Кармайкла**, их бесконечное множество, наименьшее число Кармайкла — 561.

Число Кармайкла - это составное число **p**, такое что $a^{n-1} \bmod p = 1$ для всех целых чисел **a**, взаимно простых с **p**.

Примеры чисел Кармайкла:

561, 1105, 1729, 2465, 2821, 6601, 8911, 10585, 15841, 29341, 41041, 46657, 52633, 62745, 63973, 75361, …

Тем не менее, тест Ферма довольно эффективен для обнаружения составных чисел.

Тест Ферма эффективен для обнаружения составных чисел при верном выборе параметра K.

Данный параметр означает количество различных чисел a из диапазона $0 < a < p$, которые необходимо перебрать для проверки числа p на простоту.

$$P\ (\text{число простое}) = 1 - \frac{1}{2^K},$$

где **K** – количество чисел **a** (кол – во итераций проверки).

При K = 1: $P\ (\text{число простое}) = 1 - \frac{1}{2^1} = \frac{1}{2}$

При K = 2: $P\ (\text{число простое}) = 1 - \frac{1}{2^2} = \frac{3}{4}$

При K = 10: $P\ (\text{число простое}) = 1 - \frac{1}{2^{10}} = \frac{1023}{1024} \approx 1$

Теорема 2.6 (Ферма). *Пусть p — простое число и $0 < a < p$. Тогда*

$$a^{p-1} \bmod p = 1.$$

```
bool ferma (long long p){
        if (p == 2)
                return true;
        srand (time (NULL));
        for (int i = 0; i < 100; i++){
                long long a = (rand() % (p - 2)) + 2;
                if (gcd (a, p) != 1)
                        return false;

                if (pows (a, p-1, p) != 1)
                        return false;
        }
        return true;
}
```

Числа **a** и **p** должны быть взаимно просты. Если это условие не выполняется, то число **p** — заведомо непростое.

Данная функция проверки использует функции нахождения НОД, а также быстрого возведения в степень по модулю.

Тест Ферма достаточно эффективен для обнаружения составных чисел при верном выборе параметра ***K***.

Однако числа Кармайкла успешно проходят тест Ферма, хотя не являются простыми.

Отчасти их можно отсечь за счёт проверки наибольшего общего делителя, но такая проверка не имеет хорошего теоретического обоснования вероятности отсечения, поэтому необходимо рассмотреть и другие тесты.

Алгоритм Евклида

Для нахождения наибольшего общего делителя можно использовать следующий алгоритм, известный как алгоритм Евклида.

Алгоритм 2.1. Алгоритм Евклида

ВХОД: Положительные целые числа a, b, $a \geq b$.
ВЫХОД: Наибольший общий делитель $\gcd(a, b)$.
1. WHILE $b \neq 0$ DO
2. $r \leftarrow a \bmod b$, $a \leftarrow b$, $b \leftarrow r$.
3. RETURN a.

Пример 2.11. Покажем, как с помощью алгоритма Евклида вычисляется $\gcd(28, 8)$:

a:	28	8	4
b:	8	4	0
r:	4	0	

Обобщенный алгоритм Евклида

Теорема 2.9. *Пусть a и b — два целых положительных числа. Тогда существуют целые (не обязательно положительные) числа x и y, такие, что*

$$ax + by = \gcd(a, b). \tag{2.13}$$

Обобщенный алгоритм Евклида служит для отыскания $\gcd(a, b)$ и x, y, удовлетворяющих (2.13). Введем три строки $U = (u_1, u_2, u_3)$, $V = (v_1, v_2, v_3)$ и $T = (t_1, t_2, t_3)$. Тогда алгоритм записывается следующим образом.

Алгоритм 2.2. Обобщенный алгоритм Евклида

ВХОД: Положительные целые числа a, b, $a \geq b$.
ВЫХОД: $\gcd(a,b)$, x, y, удовлетворяющие (2.13).

1. $U \leftarrow (a, 1, 0)$, $V \leftarrow (b, 0, 1)$.
2. WHILE $v_1 \neq 0$ DO
3. $q \leftarrow u_1 \text{ div } v_1$;
4. $T \leftarrow (u_1 \bmod v_1, u_2 - qv_2, u_3 - qv_3)$;
5. $U \leftarrow V$, $V \leftarrow T$.
6. RETURN $U = (\gcd(a,b), x, y)$.

Результат содержится в строке U.

Операция div в алгоритме — это целочисленное деление

$$a \text{ div } b = \lfloor a/b \rfloor.$$

Пример 2.12. Пусть a=28, b=19. Найдем числа x и y, удовлетворяющие (2.13).

U				28	1	0	
V	U			19	0	1	
T	V	U		9	1	-1	$q = 1$
	T	V	U	1	-2	3	$q = 2$
		T	V	0	19	-28	$q = 9$

Методы взлома шифров, основанных на дискретном логарифмировании

Вычисления параметров во многих криптографических шифрах основываются на односторонних функциях.

$$y = a^x \bmod p$$

Обратная функция (дискретный логарифм) обозначается:

$$\mathrm{x} = \log_a y \bmod p$$

Можно вычислить ***y*** , если даны ***a*** и ***x***, затратив не более, чем **2 log *x*** операций.

Однако отыскание ***x*** по известным ***a*** и ***y*** , т.е. вычисление дискретного логарифма, — задача намного более трудная.

Достаточно рассматривать только показатели x, удовлетворяющие неравенству $0 \le x \le p-1$.

Обозначим через t_y число операций умножения, необходимых для вычисления y в (3.1) по a и x, и будем для краткости называть t_y временем вычисления. Время возведения в степень по алгоритмам из разд. 2.1 не больше $2\log x$, причем $x < p$. Отсюда

$$t_y \le 2\log p \qquad (3.2)$$

при любом показателе степени x.

Задача отыскания x в (3.1) по данным a и y.

Оценим сложность прямого перебора:

Сначала вычислить $\mathbf{a^1}$ и проверить, верно ли равенство $\mathbf{a^1 = y}$.
Если нет, то проверяем $\mathbf{a^2 = y}$, если нет, то $\mathbf{a^3 = y}$ и т.д. до $\mathbf{a^{p-1}}$.

В среднем потребуется **(p - 1) / 2** раз умножать на **a** и проверять равенство. Таким образом, время прямого перебора:

$$t_{\text{п.п.}} \approx p/2.$$

В методе «шаг младенца, шаг великана» время отыскания **x** существенно меньше:

$$t_{\text{ш.м.ш.в.}} \approx 2 \cdot \sqrt{p},$$

а в методе исчисления порядка это время еще меньше:

$$t_{\text{и.п.}} \approx c_1 \cdot 2^{c_2 \sqrt{\log p \log \log p}},$$

где c_1 , c_2 — некоторые положительные константы.

Чтобы сделать сравнение более наглядным, выразим время вычисления через длину числа p в (3.1). Обозначим эту длину в битах через n. При вычислениях по модулю p имеем $n \approx \log p$. Поэтому порядок трудоемкости (в смысле количества операций) упомянутых алгоритмов будет следующий:

$$\begin{aligned} t_y &\approx n, \\ t_{\text{п.п.}} &\approx 2^{n-1}, \\ t_{\text{ш.м.ш.в.}} &\approx 2^{n/2}, \\ t_{\text{и.п.}} &\approx 2^{c_2\sqrt{n \log n}}, \end{aligned}$$

где $\approx$ означает «пропорционально».

Метод «шаг младенца, шаг великана»

В открытой литературе этот метод был впервые описан **Дэниелом Шенксом** (Daniel Shanks) в 1973 году.

Это один из первых методов, который показал, что задача вычисления дискретного логарифма может быть решена значительно быстрее, чем методом перебора.

Описание метода «шаг младенца, шаг великана»:

Шаг 1. Сначала берем два целых числа m и k, такие, что

$$mk > p. \tag{3.3}$$

Шаг 2. Вычислим два ряда чисел

$$y, \quad ay, \quad a^2y, \quad \ldots, \quad a^{m-1}y \pmod{p}; \tag{3.4}$$

$$a^m, \quad a^{2m}, \quad \ldots, \quad a^{km} \pmod{p} \tag{3.5}$$

(все вычисления проводятся по модулю p).

Шаг 3. Найдем такие i и j, для которых выполняется равенство

$$a^{im} = a^j y. \tag{3.6}$$

Утверждение 3.1. *Число*

$$x = im - j \tag{3.7}$$

является решением уравнения (3.1). *Кроме того, целые числа* i, j, *удовлетворяющие* (3.6), *существуют.*

Доказательство. Справедливость (3.7) следует из приводимой ниже символической цепочки равенств, где все вычисления даны по модулю p, а деление соответствует умножению на обратный элемент:

$$a^x = a^{im-j} = \frac{a^{im}}{a^j} = \frac{a^{im}y}{a^j y} = \frac{a^{im}y}{a^{im}} = y.$$

Докажем теперь, что числа i и j, удовлетворяющие (3.6), существуют. Для этого сведем все числа вида (3.7) в таблицу 3.1.

Таблица 3.1. **Распределение чисел вида** $im - j$

$i \downarrow \quad j \rightarrow$	0	1	2	...	$m-1$
1	m	$m-1$	$m-2$	...	1
2	$2m$	$2m-1$	$2m-2$	...	$m+1$
...	...	...	...	...	...
k	km	$km-1$	$km-2$	...	$(k-1)m+1$

Мы видим, что в таблице содержатся все числа от 1 до km. Значит, из (3.3) следует, что в таблице содержатся все числа от 1 до p. Таким образом, любой показатель степени $x < p$ будет содержаться в таблице, т.е. число x, удовлетворяющее (3.1), может быть представлено в виде (3.7) и всегда найдется в таблице, поэтому уравнение (3.6) всегда имеет решение.

Пример 3.1. Найдем решение уравнения $2^x \bmod 23 = 9$, используя метод «шаг младенца, шаг великана».

Выберем m, k. Пусть $m = 6$, $k = 4$. Мы видим, что (3.3) выполняется. Вычислим числа (3.4), (3.5):

$$(3.4): \quad 9, 18, 13, 3, 6, 12;$$
$$(3.5): \quad 18.$$

Дальнейшие вычисления не проводим, так как уже нашлись одинаковые числа в (3.4) и (3.5) при $i = 1$, $j = 1$. По (3.7) получаем

$$x = 1 \cdot 6 - 1 = 5.$$

Проверим: $2^5 \bmod 23 = 9$. Действительно, $x = 5$ есть решение.

Имея два описанных ранее ряда, можно быстро найти в них равные элементы, воспользовавшись одним из описанных ниже методов.

Метод с использованием сортировки:

Пусть есть два массива $A[m], B[k]$ состоящие соответственно из m и k элементов. Ниже представлен код на языке C++, который позволяет находить в этих массивах равные элементы с трудоемкостью $O(m \log_2 m + k \log_2 k)$.

```
int A[m], B[k];
sort(A,A+m);
sort(B,B+k);
int i=0, j=0;
while(i<m && j<k)
{
  if(A[i]<B[j])
    ++i;
  else if(A[i]>B[j])
    ++j;
  else break;
}
if(i<m && j<k)
  cout << i << " " << j;
```

Метод с использованием структур данных:

Пусть есть два массива *A[m], B[k]* состоящие соответственно из m и k элементов. Ниже представлен код на языке C++, который позволяет находить равные элементы с трудоемкостью $O(m\log_2 m + k\log_2 k)$. Либо, в зависимости от используемой структуры данных, с трудоемкостью $O(mC + kC)$, где C - некоторая константа.

```
int A[m], B[k];
map<int,int> dict;
for(int i=0; i<m; ++i)
  dict[A[i]]=i;
for(int i=0; i<k; ++i)
  if(dict.count(B[i]))
  {
    cout << i << " " << dict[B[i]];
    break;
  }
```

Утверждение 3.2. *Время вычислений по данному методу при больших p удовлетворяет неравенству*

$$t_{ш.м.ш.в.} \leq \text{const} \cdot \sqrt{p} \log^2 p. \tag{3.8}$$

(Здесь речь идет о полном времени вычислений, а не о числе умножений).

Доказательство. Мы можем взять

$$k = m = \lfloor \sqrt{p} \rfloor + 1, \tag{3.9}$$

так что, очевидно, (3.3) выполняется. Тогда в (3.4) и (3.5) потребуется не более $2\sqrt{p}$ операций умножения. Мы знаем, что для «обычных» («школьных») методов умножения и деления время вычисления результата для двух r-значных чисел пропорционально r^2. У нас все числа берутся из множества $\{1, \ldots, p\}$, значит, $r \leq \log p$, и время вычисления пропорционально $\log^2 p$. Отсюда мы сразу получаем время, затраченное на вычисление рядов (3.4) и (3.5). Однако мы не

учли все этапы алгоритма. Мы не учли время, требуемое для нахождения равных чисел в этих рядах. При больших k и m это далеко не простая задача. Она может быть решена следующим образом: сначала каждому числу припишем его номер в ряду и еще один бит, в котором указана принадлежность к ряду (3.4) или (3.5), затем преобразуем обе последовательности в список и отсортируем (упорядочим по величине). Длина общего ряда равна $k + m \approx 2\sqrt{p}$. Для лучших методов сортировки требуется $S \log S$ операций сравнения, где S — число элементов в списке (см., например, [1]). В нашем случае $S = 2\sqrt{p}$ и, следовательно, требуется $2\sqrt{p} \log \left(2\sqrt{p}\right) \approx \sqrt{p} \log p$ операций сравнения над словами длины $\log p$ бит, т.е. всего требуется порядка $\sqrt{p} \log^2 p$ операций. После сортировки объединенного ряда его надо просмотреть и найти два равных числа из разных рядов (3.4), (3.5), используя битовый признак. Таким образом, суммируя время вычисления на всех этапах, получаем (3.8).

Шифр

- Имеет ключ
- Подчиняется принципам Керкхоффса
- Шифрование != кодирование

Виды атак

- **По шифротексту.**
- **По известному тексту**. Это происходит, если Ева получает в свое распоряжение какие-либо открытые тексты, соответствующие ранее переданным зашифрованным. Сопоставляя пары «текст-шифротекст», Ева пытается узнать секретный ключ, чтобы с его помощью дешифровать все последующие сообщения от Алисы к Бобу.
- **По выбранному тексту**. Противник пользуется не только предоставленными ему парами «текст-шифротекст», но может и сам формировать нужные ему тексты и шифровать их с помощью того ключа, который он хочет узнать.

Принципы Керкхоффса (1883 г.)

1. Система должна быть физически, если не математически, невскрываемой.
2. Нужно, чтобы не требовалось сохранение системы в тайне; попадание системы в руки врага не должно причинять неудобств.
3. Хранение и передача ключа должны быть осуществимы без помощи бумажных записей; корреспонденты должны располагать возможностью менять ключ по своему усмотрению.

Принципы Керкхоффса (1883 г.)

4. Система должна быть пригодной для сообщения через телеграф (в настоящее время – по сети).
5. Система должна быть легко переносимой, работа с ней не должна требовать участия нескольких лиц одновременно.
6. От системы требуется, учитывая возможные обстоятельства её применения, чтобы она была проста в использовании, не требовала значительного умственного напряжения или соблюдения большого количества правил.

Быстрое возведение в степень

В качестве примера односторонней функции рассмотрим следующую:

$$y = a^x \bmod p, \tag{2.3}$$

где p — некоторое простое число (т.е. такое, которое делится без остатка только на себя и на единицу), а x — целое число из множества $\{1, 2, \ldots, p-1\}$.

Обратная функция обозначается

$$x = \log_a y \bmod p \tag{2.4}$$

и называется *дискретным логарифмом*.

Утверждение 2.1 (о сложности вычислений (2.3)). *Количество операций умножения при вычислении* (2.3) *по описанному методу не превосходит* $2\log x$.

Доказательство. Для вычисления чисел ряда (2.5) требуется t умножений, для вычисления y по (2.6) не более, чем t умножений (см. пример 2.1). Из условия $t = \lfloor \log x \rfloor$, учитывая, что $\lfloor \log x \rfloor \leq \log x$, делаем вывод о справедливости доказываемого утверждения. □

Замечание. Как будет показано в дальнейшем, при возведении в степень по модулю p имеет смысл использовать только показатели $x < p$. В этом случае мы можем сказать, что количество операций умножения при вычислении (2.3) не превосходит $2\log p$.

Важно отметить, что столь же эффективные алгоритмы вычисления обратной функции (2.4) неизвестны. Один из методов вычисления (2.4) называется «шаг младенца, шаг великана». Этот метод требует порядка $2\sqrt{p}$ операций.

Покажем, что при больших p функция (2.3) действительно односторонняя, если для вычисления обратной функции используется метод «шаг младенца, шаг великана. Получаем следующий результат (табл. 2.1).

Таблица 2.1. **Количество умножений для вычисления прямой и обратной функции**

Количество десятичных знаков в записи p	Вычисление (2.3) ($2\log p$ умножений)	Вычисление (2.4) ($2\sqrt{p}$ умножений)
12	$2 \cdot 40 = 80$	$2 \cdot 10^6$
60	$2 \cdot 200 = 400$	$2 \cdot 10^{30}$
90	$2 \cdot 300 = 600$	$2 \cdot 10^{45}$

Мы видим, что если использовать модули, состоящие из 50–100 десятичных цифр, то «прямая» функция вычисляется быстро, а обратная практически не вычислима. Рассмотрим, например, суперкомпьютер, который умножает два 90-значных числа за 10^{-14} сек. (для современных компьютеров это пока не доступно). Для вычисления (2.3) такому компьютеру потребуется

$$T_{\text{выч.пр.}} = 600 \cdot 10^{-14} = 6 \cdot 10^{-12} \text{ сек.},$$

а для вычисления (2.4) —

$$T_{\text{выч.обр.}} = 10^{45} \cdot 10^{-14} = 10^{31} \text{ сек.},$$

т.е. более 10^{22} лет.